

A Survey on Graph Neural Networks for Intrusion Detection Systems: Methods, Trends and Challenges

Meihui Zhong^a, Minwei Lin^{a,*}, Chao Zhang^b and Zeshui Xu^c

^aCollege of Computer and Cyber Security, Fujian Normal University, Fuzhou, 350117, Fujian, China

^bSchool of Computer and Information Technology, Key Laboratory of Computational Intelligence and Chinese Information Processing of Ministry of Education, Shanxi University, Taiyuan, 030006, Shanxi, China

^cBusiness School, Sichuan University, Chengdu, 610064, Sichuan, China

ARTICLE INFO

Keywords:

Intrusion Detection Systems
Graph Neural Networks
Graph Representation Learning
Information Retrieval
Anomaly Detection

ABSTRACT

Intrusion detection systems (IDS) play a crucial role in maintaining network security. With the increasing sophistication of cyber attack methods, traditional detection approaches are encountering more challenges. In recent years, graph neural networks (GNNs) have garnered significant attention in the field of intrusion detection due to their unique ability to capture the relationships within the graph structure of data communications. In this review, we propose a novel taxonomy that categorizes advanced research into three distinct areas: tasks related to graph construction, network design, and GNN models deployment. We detail a generalized design process for GNN-based intrusion detection models, discussing the challenges encountered at each stage. Building upon these discussions, we conduct a systematic survey of existing works. Ultimately, we delve into a thorough exploration of the future research directions and the pending issues in this domain. By adopting a problem-oriented taxonomy and conducting a targeted survey, this review aims to provide scholars with a clear, systematic framework for deepening their understanding and further exploration of the field.

1. Introduction

Intrusion detection is a critical component of maintaining cyber security. Its purpose is to identify threats while preserving the integrity and performance of network deployment. It dynamically detects various attacks from both internal and external sources in real-time, ensuring timely, accurate, and comprehensive detection of intrusions. With the accelerated pace of global digitization, the frequency and complexity of cyber attacks have dramatically increased (Landauer et al., 2023; Javed et al., 2023). These attacks not only compromise the data security of individuals and businesses but can also result in substantial economic losses and harm to reputation. Given the severity of these threats, there is a pressing need for more sophisticated intrusion detection technologies to counteract and neutralize them.

Deep learning techniques, notably in image recognition and natural language processing, have shown remarkable results (Pouyanfar et al., 2018). In the field of intrusion detection, researchers are exploring machine learning techniques like convolutional neural networks (CNNs) and recurrent neural networks (RNNs). They treat intrusion detection as a feature learning problem (Yan et al., 2023), an approach illustrated in Fig. 1 (a). They attempt to automatically learn meaningful features from raw communication information. These deep learning models have made significant advancements in domains characterized by large-scale and high-dimensional data, such as network traffic analysis and event correlation in cyber security (Basati and Faghih, 2022; Kaur et al., 2023).

However, network traffic and system data are highly structured in nature, embodying rich relationships among various elements such as IP addresses, protocols, and host events. Traditional deep learning models struggle to capture these intricate relationships effectively (Cui et al., 2019; Huang et al., 2021; Guo et al., 2023). For intrusion detection, incessant refinements to conventional network structures yield marginal improvements at best. Graph neural networks (GNNs), a subfield of deep learning, leverage the strengths of traditional CNNs within explicit and implicit graph structures (Chen and Wu, 2022). By modeling data like network traffic, communication patterns, and system logs as graphs, GNNs can effectively capture the intricate interactions and relationships embedded within, as shown in Fig. 1(b). Furthermore, GNNs are capable of identifying not only direct relationships between entities but also higher-order connections, like multi-hop context information (Gao et al., 2023; Sun et al., 2023, 2021). This capability enables GNNs to excel in intrusion detection, especially in identifying complex attack patterns.

The application of GNNs in intrusion detection systems presents distinct challenges. To fully harness the capabilities of GNNs in intrusion detection, several challenges must be addressed. First, constructing an appropriate graph structure from raw data is crucial. Data in the intrusion detection domain, such as network traffic or system data, does not inherently possess a graph structure, necessitating processing to render it compatible with GNNs. Second, designing an effective GNN structure to analyze these graphs and extract meaningful features is challenging. Furthermore, given the efficiency and computational cost requirements of intrusion

*Corresponding author

✉ mhzhong2021@163.com (M. Zhong); linmwcs@163.com (M. Lin);
mailto:czhang@sxu.edu.cn (C. Zhang); xuzeshui@263.net (Z. Xu)
ORCID(s): 0000-0003-2026-7178 (M. Lin)

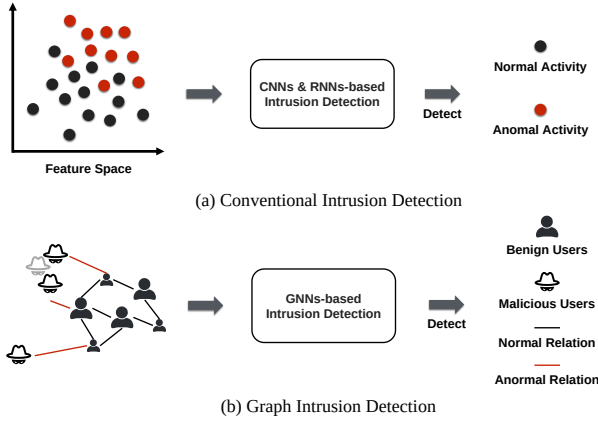


Figure 1: Toy Samples of (a) Conventional Intrusion Detection and (b) Graph Intrusion Detection. Apart from anomalies shown in (b), GNN-based intrusion detection also identifies graph-level anomalies, detailed in Sections 4.2.3.

detection, feature extraction from graph structures incurs extra computational expenses. Hence, the efficient deployment and execution of these models are focal points of research.

In this review, we summarize existing GNN works in intrusion detection, delving into the design process and associated challenges. We present a taxonomy that classifies GNN-based intrusion detection into three main categories: graph construction tasks, graph analysis tasks, and GNN model deployment methods. This taxonomy, as depicted in Fig. 2, underpins our comprehensive survey of the current literature, offering a structured understanding of the field. We hope that this problem-oriented taxonomy provides readers with a clear, systematic framework, aiding them in gaining a deep understanding of the latest advancements in this domain.

There are some differences between this survey and the existing ones. It is worth mentioning that Bilot et al. (2023) presented the first overview of GNN-based intrusion detection and have made significant contributions in this area. Our work differs from theirs in several key aspects, the first of which is the taxonomy. They primarily focus on network intrusion detection and host intrusion detection, then categorize based on the type of graph. Their work then showcases various classification levels and different graph neural network models. This classification is distinct from ours, and the two cannot be directly compared side-by-side. Secondly, they overlook the deployment methods of important GNN networks, especially the distributed GNN models, which are of significant interest in the industry. Thirdly, the rapid development of this field means that some recent, significant literature was not included in their previous survey.

While only a few reviews specifically concentrate on GNN applications in intrusion detection, many others address closely related topics. For instance, Weller-Fahy et al. (2015) explored network intrusion detection using similarity

metrics, and Liu et al. (2018) reviewed host intrusion detection with a focus on system call graphs. Buczak and Guven (2016) provided an extensive survey on machine learning and data mining methods in this field, emphasizing methodological complexities and challenges. Umer et al. (2017) detailed flow-based intrusion detection, including model architectures and datasets. Gibert et al. (2020) discussed machine learning techniques in malware detection, covering workflows and recent trends. Jamalipour and Murali (2022) and Nguyen and Reddi (2023) examined intrusion detection techniques in IoT, with the former encompassing machine learning, deep learning, and reinforcement learning, and the latter emphasizing reinforcement learning methods. Corona et al. (2013) offered a detailed analysis of adversarial attacks against IDSs, providing insights into vulnerabilities and attack tactics. Dong et al. (2023) concentrated on investigating the advancements of graph neural networks in IoT. The tasks in this field encompass intrusion detection, performance diagnosis, time series forecasting, and environmental monitoring. They found that applying GNNs in IoT intrusion detection is an emerging trend.

To our knowledge, although there are some interesting surveys in both GNN (Zhou et al., 2020a; Wu et al., 2021; Zhang et al., 2022c; Yuan et al., 2023) and intrusion detection, the interplay between GNN and intrusion detection data still needs further exploration. Overall, our survey holds significant distinctions from these surveys.

The contributions of this survey are as follows:

- 1) **Comprehensive Review:** We extensively discuss existing literature, thoroughly exploring the interactions between graph neural network and intrusion detection. We present a universal design process, providing insights into each step.
- 2) **New Taxonomy:** We propose a systematic taxonomy for consistently classifying the current and future research on GNN-based intrusion detection methods. We analyze the primary research within each category along with their methodologies.
- 3) **Survey Objective:** We define four research questions (RQs), with the aim of addressing the concerns prevalent in the field through a comprehensive review.
- 4) **Future Directions:** We discuss the limitations of current methods and present several potential future directions.

The remainder of this study is organized as follows: Section 2 introduces the fundamentals of intrusion detection and graph neural networks, emphasizing the motivation behind employing graph neural networks for intrusion detection. Section 3 discusses the challenges associated with applying graph neural networks in intrusion detection. Subsequently, Section 4 categorizes existing methods based on these challenges and presents a detailed discussion centered around four RQs, drawing upon the survey of existing methods. Finally, Section 5 explores the current challenges in this field and proposes potential directions for future research.

Table 1
Summary and Comparison of Related Studies.

Surveys	Year	Types of Analyzed Data	Survey Content		
Weller-Fahy et al. (2015)	2015	Network-IDS	Distance and similarity measures		
Buczak and Guven (2016)	2016	Network-IDS Host-IDS	Machine learning	Data mining	Cyber datasets
Umer et al. (2017)	2017	Network-IDS	Machine learning	Statistical techniques	Flow-based datasets
Gibert et al. (2020)	2019	Malware detection	Machine learning	Deep learning	
Liu et al. (2018)	2019	Host-IDS with system calls	Data mining	Host-datasets	Application on systems
Jamalipour and Murali (2022)	2021	Network-IDS	Machine learning	Deep learning	Reinforcement learning
Nguyen and Reddi (2023)	2021	Network-IDS Host-IDS	Deep reinforcement learning		
Bilot et al. (2023)	2022	Network-IDS Host-IDS	Graph neural network	Network and host-based datasets	
Dong et al. (2023)	2023	IoT sensing environments	Graph neural network	IoT sensing datasets	
Our Survey	2023	Network-IDS Host-IDS	Graph neural network	Graph construction	GNN model deployment

2. Background

In order to make the presentation of the article more comprehensive, we will introduce the background of intrusion detection and graph neural networks in this section.

2.1. Intrusion Detection

The intrusion detection system plays a crucial role in the field of network security (Li et al., 2021b). It continuously monitors and analyzes computer systems and networks to detect signs of unauthorized access or malicious activities. Its primary goal is to ensure the security, integrity, and availability of digital assets by quickly identifying and responding.

From a data source perspective, intrusion detection systems can be divided into network intrusion detection systems (Network-IDS) and host intrusion detection systems (Host-IDS) (Debar et al., 1999). The former detects potential threats by monitoring network traffic, while the latter focuses on applications and processes running on a single host. Host-IDS can analyze system calls, file activities, and logs more deeply (Pontarelli et al., 2013; Creech and Hu, 2014).

The effectiveness of IDS is often determined by their detection methods. Signature-based detection relies on pre-defined patterns of known malicious activities. While it is effective against known threats, it may overlook new ones. In contrast, anomaly-based detection establishes a baseline of normal system behavior and flags deviations. It has the potential to identify new threats but might also result in false alarms (Ye et al., 2002; Hwang et al., 2007).

Essentially, its detection process can be defined as two sub-problems: determining whether the current data being detected is normal or anomalous. Such automated systems can typically be implemented using machine learning methods. Modern intrusion detection mechanisms can roughly be divided into three categories: methods based on shallow models (Jain et al., 2022; Gavel et al., 2022; Fraihat et al., 2023), methods based on deep models (Singh et al., 2023; Abdel Wahab, 2022; Wu et al., 2023a), and methods based on ensemble models (Lazzarini et al., 2023; Ren et al., 2023; Louk and Tama, 2023).

To harness vast amounts of data and transform it into valuable knowledge inputs, thereby enhancing the detection capabilities against cyber attacks, researchers continuously integrate machine learning and deep learning techniques to improve network intrusion detection. While convolutional neural networks (CNNs) and recurrent neural networks (RNNs) have gained significant advantages in processing data within flat Euclidean spaces, these models lose their inherent strengths when faced with data presented in high-dimensional manifolds (Han et al., 2019; Zhong et al., 2023). The issue lies in two main aspects. Firstly, the intricate patterns and interactions among data are challenging to describe using sequences or grids, rendering traditional deep learning models incapable of handling such non-Euclidean structures. Secondly, deep neural networks often rely on the availability of vast amounts of costly labeled data (Qi and Luo, 2022; King and Huang, 2023). However, data collected from various devices is often influenced by factors such as network latency and packet loss. This results in the performance of data-driven deep learning models being constrained (Qian et al., 2022).

2.2. Graph Neural Network

Recently, with the advancements in CNNs and graph representation learning (GRL) (Zhou et al., 2020a; Wu et al., 2021), researchers have introduced graph neural networks (GNNs). Unlike traditional neural networks, GNNs are specifically designed to handle graph-structured data, effectively capturing the intricate relationships between nodes and edges. In this section, we first introduce the types of graphs, followed by a brief overview of the design of GNN models.

2.2.1. Graph Type

The graph is a fundamental concept in mathematics and computer science used to represent relationships between objects. In graph theory, a graph is typically composed of nodes (or vertices) and edges. A graph $\mathcal{G}$ can be defined as an ordered pair $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a non-empty finite set representing the nodes in the graph, and $\mathcal{E}$ is a finite set representing the edges between nodes. Let $v_i \in \mathcal{V}$ be a node

and $e_{i,j} \in \mathcal{E}$ be an edge. The neighbors of the node v_i are represented as $\mathcal{N}_{v_i} = \{v_j \in \mathcal{V} | e_{i,j} \in \mathcal{E}\}$.

In the field of intrusion detection, understanding the type of graphs utilized can enhance the modeling and analysis process. We categorize the types of graphs into two: static graphs and dynamic graphs. Static graphs may be more suitable for offline analysis, vulnerability assessment, or historical data analysis. In contrast, dynamic graphs are essential for real-time monitoring and threat detection in continuously evolving network environments.

- **Static Graph.** Static graphs are often used to represent the steady-state structure of a network or host. They are particularly useful for modeling stable relationships and interactions within a network, providing a foundational understanding of its normal behavior. However, static graphs are not adept at capturing temporal dynamics and evolving relationships within a network, which are often crucial for identifying and understanding sophisticated cyber threats.

- **Dynamic Graph.** Dynamic graphs are essential for modeling the temporal aspects of network traffic and system behavior, providing a more comprehensive view of potential security threats. They are particularly relevant for capturing the intricacies of advanced persistent threats (APTs) and other sophisticated cyber attacks that unfold over time (Alshamrani et al., 2019). The analysis of dynamic graphs is computationally more intensive due to their changing structure, requiring advanced modeling techniques and algorithms to effectively extract meaningful insights.

Furthermore, based on whether the edges carry a weight, graphs can be further divided into weighted and unweighted graphs. In weighted graphs, each edge has an associated weight value.

2.2.2. Network Design

The core idea behind GNNs is to update the representation of a node by fusing information from its neighbors. In this section, we introduce the general framework of graph neural networks (Xu et al., 2019). Starting with the initial node representation $\mathbf{h}^0 = \mathbf{x}$, there are two crucial functions implemented at each layer to facilitate the message-passing process:

- **AGGREGATE**, which is responsible for **aggregate** messages based on the current representation of a node and its neighbors.

- **UPDATE**, which is used to compute the new feature of a node based on its current feature and the incoming messages.

Mathematically, we can define the general framework of graph neural networks as follows:

Initialization: $\mathbf{h}^0 = \mathbf{x}$. For $l = 1, 2, \dots, L$,

$$\begin{aligned} \mathbf{n}_{v_i}^{(l)} &= \text{AGGREGATE}_l \left(\left\{ \mathbf{h}_{v_j}^l, \forall v_j \in \mathcal{N}_{v_i} \right\} \right), \\ \mathbf{h}_{v_i}^{(l+1)} &= \text{UPDATE}_l \left(\mathbf{h}_{v_i}^{(l)}, \mathbf{n}_{v_i}^{(l)} \right), \end{aligned} \quad (1)$$

where $\mathbf{n}_{v_i}^{(l)}$ represents the aggregated representation of the neighboring nodes of node v_i , and $\mathbf{h}_{v_i}^{(l)}$ represents the representation of node v_i at l th layer.

When we input graph structure and node content information into graph neural networks, the output can be tailored to address different graph analysis tasks by focusing on various mechanisms:

- **Node-Level Task**, which tasks involve making predictions or classifications for individual nodes in the graph. The node-level task primarily utilizes the final node representations after the message-passing process, i.e., $\mathbf{h}_{v_i}^{(L)}$ for each node v_i .

- **Edge-Level Task**, which tasks focus on predictions related to the edges or connections between nodes. For this task, GNNs often need to compute representations for edges based on the representations of the connected nodes.

- **Graph-Level Task**, which tasks involve making predictions for the entire graph. This tasks often require a readout function to aggregate information from all nodes and edges in the graph to produce a graph-level representation. Graph-level analysis involves examining the entire graph structure to take into account the current overall environment and identify various types of attacks.

Each of these tasks requires a different design and training approach for the GNN model. For node-level and edge-level tasks, the loss function is usually defined based on the predictions for individual nodes or edges. In contrast, for graph-level tasks, the loss function is defined based on the predictions for entire graphs. The choice of AGGREGATE and UPDATE functions, as well as the depth of the network (i.e., the number of layers L), also need to be carefully selected based on the specific task at hand.

2.3. Why Use GNNs for Intrusion Detection?

Before delving into the intricacies of recent advancements, it is crucial to grasp the rationale behind the adoption of graph neural networks (GNNs) in intrusion detection. GNNs have changed the detection landscape by revealing latent patterns within data, thereby elevating intrusion detection systems (IDS) performance. The advantages of GNNs in this context can be examined from three perspectives: (1) Structural property of data; (2) Higher-order interaction; (3) Behavioral supervisory signal.

Structural Property of Data: Data, whether sourced from network packets as traffic or from detailed protocol decoding as host events and logs, represents semi-structured information. While these data have no explicitly graph structure, they are inherently interconnected. Traditional detection models often neglect these structural relationships (Duan et al., 2023), but GNNs excel in discerning and learning from them. As a result, GNNs produce high-quality data embeddings, which are crucial for optimal detection performance.

Higher-Order Interaction: GNNs facilitate the flow of information across multi-hop nodes, fostering interactions that extend beyond direct connections through their unique propagation mechanisms (Cui et al., 2019). For instance,

once malicious software infiltrates a host system, the entire environment becomes vulnerable to subsequent attacks. Nodes that exhibit similar patterns of information propagation are likely to be compromised, an effect known as collaborative filtering. Standard neural networks, such as CNNs and RNNs, cannot handle the connections between input data nor capture the potential distribution structure of the data. This is because they process node features as isolated inputs without considering the structural similarities between data. Their primary limitation is their focus on first-order interactions and isolated data analysis. High-order interactions represent a critical pattern for the transmission of information in intrusion detection. The emergence of GNNs allows models to capture these hidden patterns more effectively.

Behavioral Supervisory Signal: The scarcity of attack instances during data collection poses a challenge for detection models. GNNs mitigate this by leveraging behavioral supervision signals within the graph structure during the learning process. As information propagates, the behavior of a labeled node can influence other unlabeled data within its scope (Liu et al., 2023). Consider the example of computer worms: the attack behavior made by a certain attacker IP is quite sparse compared to its normal behavior. If detection models focus solely on these infrequent attack behaviors, they may suffer from a high false negative rate. However, GNN-based models can effectively improve by incorporating additional context (Yu et al., 2023), such as the IP's non-target behavior of repeatedly scanning other IPs within a certain timeframe. These models can significantly enhance the performance of intrusion detection systems in predicting attack behaviors.

3. Challenges of Applying GNNs to Intrusion Detection

We propose a novel problem-oriented taxonomy to categorize existing GNN-based models and provide an in-depth discussion about them. Specifically, to effectively utilize GNNs for intrusion detection, there are several crucial challenges that researchers need to address, which are as follows:

- 1) How can cyber security data be effectively adapted to a graph structure?
- 2) What features should be learned from the graph structure?
- 3) How should GNN models be deployed?

In the following, we will elaborate on these three challenges one-by-one.

3.1. Graph Construction

Before diving into the intricacies of graph-based intrusion detection, it is essential to understand how data, especially in the realm of cyber security, can be transformed into graph structures. This transformation is crucial as it allows for the application of graph-based algorithms and

methodologies to datasets that were traditionally not viewed in a graph context. While network traffic and host data are inherently structured, they mostly exist as implicit graph structures. We focus on how to transform conventional network data into a graph structure. This encompasses choosing appropriate representations for nodes and edges, as well as effectively capturing relationships and patterns within the network.

A well-designed graph structure can reveal more high-quality information about the data. For intrusion detection systems, input data requires meticulous design to form informative graph structures. For instance, nodes represent hosts, and edges represent relationships. The solutions go beyond these, with the key focus on constructing graphs that capture complex interactions between data, such as network traffic or host log information.

3.2. Network Design

A well-designed GNN model can learn higher-quality information, leading to superior performance in subsequent detection tasks. Existing studies focus on network structure design, including propagation and aggregation methods, each with their own advantages and disadvantages. Regarding detection tasks, the emphasis lies in designing GNN structures to learn higher quality graph representations while maintaining a balance between model complexity and detection performance.

GNNs have a myriad of hyperparameters, such as the count of neurons in each network layer and the count of neighboring nodes. Standard GNN architectures are designed to learn features under ideal conditions, where the observed graph data (structure, attributes, labels) is sufficiently complete to provide ample information. However, in network or host environments, there is a high likelihood of packet loss, IP address spoofing, or insufficient labeled data. The question of how to derive a strong model from weak information represents a pivotal research challenge in model architecture design (Liu et al., 2023).

3.3. GNN Model Deployment

While GNNs can extract richer structural features distinct from those obtained by CNNs or traditional machine learning methods, they come at a significantly higher computational cost (Cai et al., 2018; Cui et al., 2019; Ma et al., 2021). Therefore, the final concern relates to the computational costs of GNNs, given the real-time nature of intrusion detection. The computational operations introduced by GNNs add to the complexity of the model, making efficient model deployment a crucial research task for intrusion detection.

Based on these challenges, existing research has been categorized into tasks related to graph construction, graph analysis, and the deployment of GNN models. Next, we provide a detailed presentation of these classifications and analyze how these studies address the aforementioned issues.

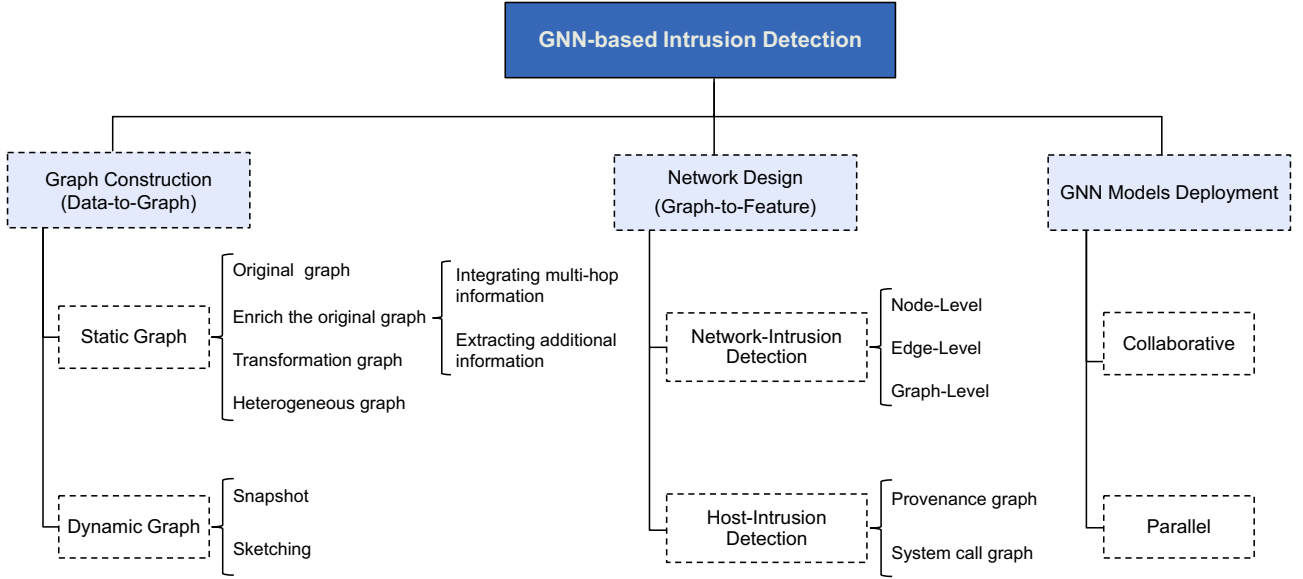


Figure 2: A Taxonomy of GNN-based Intrusion Detection.

4. Existing Research

Recent advancements in graph-based methodologies have underscored their efficacy in diverse domains, notably within the realm of intrusion detection, as delineated in Section 2.3. However, there are numerous challenges that arise during the design process of these methods, outlined in Section 3. As such, we have proposed a problem-oriented taxonomy that systematically organizes the body of existing research. Fig. 2 provides a visual overview of the different categories and subcategories within GNN-based intrusion detection. We have delineated the main categories according to three distinct design phases: data to graph transformation, graph to feature extraction, and GNN models deployment. The subcategories within this framework will be discussed in detail in the corresponding subsections.

4.1. Data-to-Graph

Graph structures excel at modeling network activities or host events, as they intuitively illustrate the structural/relational information between different entities or events (Ma et al., 2021). To represent traffic or log data with graphs, it is imperative first to extract relevant data from the dataset and transform it into graph components based on the entities involved. Once the raw data is stored in the graph, various tools and techniques can be utilised to analyze the graph data. This approach potentially offers a powerful alternative to traditional intrusion detection, providing deep insights into the behaviors among entities and identifying anomalies (Messai and Seba, 2023). The conventional method of graph construction involves using IP addresses as nodes in the graph, with data packets serving as the edges between nodes. Several issues arise from this: (1) GNNs need to learn edge embeddings to classify data packets, but does learning node embeddings not matter? (2) Considering that exploring

different types of embeddings or incorporating additional node and edge attributes might enhance the graph's representational power, should GNNs learn other hidden information within the graph? (3) Considering computational efficiency and model performance, should operations be performed on the entire graph, or should sampling be conducted on representative subgraphs?

With these questions in mind, we delved deeply into the various existing methods for constructing graph structures. The various methods are summarized in Table 2, and we have categorized them into two main types: static graphs (Section 4.1.1) and dynamic graphs (Section 4.1.2).

4.1.1. Static Graph

Most studies (Zhou et al., 2020b; Wang et al., 2020; Protogerou et al., 2021; Lo et al., 2022; Lan et al., 2022; Caville et al., 2022; Li et al., 2023; Ko et al., 2023; Xu et al., 2023; Agiollo et al., 2023; Hu et al., 2023; Wei et al., 2021; Wang et al., 2022; Wu et al., 2023b; Hu et al., 2021) utilized the original graph construction methods. For Network-IDS, this typically involves representing IP addresses as nodes and the interactions or connections represented by data packets as edges. For Host-IDS, it involves explicit graph structures, such as provenance graphs and system call graphs. Applying GNNs on these original graphs presents two main issues: compatibility and effectiveness. The fixed patterns of original graph structures often limit analysis to edge classification, where current GNN models underperform due to inadequate utilization of edge features (Gong and Cheng, 2019).

One strategy to address the first issue is to highlight different information through graph transformation. Some studies have transformed graph structures into line graphs, which allow for the simultaneous use of node and edge features and convert edge classification tasks into node classification

Table 2
Summary of GNN-based Intrusion Detection.

Analyzed Data Types	Graph Types	Graph Construction Methods	Network Design	Graph Embedding Methods	Method	Year
Network-IDS	Static Graph	Original graph	GCN	Node-level	(Zhou et al., 2020b)	2020
			Statistical Analysis	Graph-level	BotMark Wang et al. (2020)	2020
			Edge/Node-MLP	Node-level	GraphNET Protogerou et al. (2021)	2021
			GraphSAGE	Edge-level	E-GraphSAGE Lo et al. (2022)	2022
			E-GraphSAGE	Edge-level	E-minBatch GraphSAGE Lan et al. (2022)	2022
			E-GraphSAGE	Edge-level	Anomal-E Caville et al. (2022)	2022
			GAT	Node-level	Li et al. (2023)	2023
			GAT	Node-level	Ko et al. (2023)	2023
			GCN	Node-level	EE-GCN Xu et al. (2023)	2023
			GIN	Node-level	GNN4IFA Agiollo et al. (2023)	2023
			Graph2Vec	Node-level	Graph2vec+RF Hu et al. (2023)	2023
		Transformation graph	E-GraphSAGE + GAT	Node-level	E-GraphSAGE & E-ResGAT Chang and Branco (2021)	2021
			GraphSAGE	Edge-level	GBA-IDS Zhu and Lu (2022)	2022
			TAGCN + Attention Mechanism	Node-level	FT-GCN Deng et al. (2023)	2023
			GCN + GAT	Node-level	Friji et al. (2023)	2023
		Enrich the original graph by integrating multi-hop information	GCN	Node-level	GCN-TC Zheng and Li (2019)	2019
			Distance-based Graph Embedding	Node-level	First-order & Second-order Graph Xiao et al. (2020)	2020
			GraphSAGE	Edge-level	TPE-NIDS Zhang et al. (2022b)	2022
		Enrich the original graph by extracting additional information	GCN	Node-level	Alert-GCN Cheng et al. (2021)	2021
			Graph Embedding	Node/Edge-level	NF-GNN Busch et al. (2021)	2021
			GCN	Node-level	Pujol-Perich et al. (2022)	2022
			GAT	Node-level	BS-GAT Wang et al. (2023)	2023
			GCN	Node-level	GID Zhang et al. (2023b)	2023
		Heterogeneous graph	CGNN	Edge-level	HetGLM Sun and Yang (2022)	2022
			GCN	Edge-level	NE-GConv Altaf et al. (2023)	2023
	Dynamic Graph	Snapshot	GCN + Attention Mechanism	Edge-level	AddGraph Zheng et al. (2019)	2019
			GCNII + GRU	Edge-level	DLGNN Duan et al. (2023)	2023
			-	Edge-level	Euler King and Huang (2023)	2023
			-	Node-level	Graph-based framework Messai and Seba (2023)	2023
		Sketching	Node2Vec	Graph-level	SNAPSKETCH Paudel and Eberle (2020)	2020
			PAGAT + WGAT	Graph-level	GLAD-PAW Wan et al. (2021)	2021
			GAT	Node-level	GLD-Net Guo et al. (2022)	2022
			Manifold Learning (UMAP)	Node-level	DMTR Zhong et al. (2023)	2023
		Provenance Graph	Word2Vec + GCN	Graph-level	DeepHunter Wei et al. (2021)	2021
			GraphSAGE	Node-level	THREATTRACE Wang et al. (2022)	2022
			Statistical Analysis (Similarity Calculation)	Node-level	Paradise Wu et al. (2023b)	2023
		System Call Graph	Word2Vec and KNN/MLP	Node-level	GRID Hu et al. (2021)	2021
Controller Area Network-IDS	Dynamic Graph	Snapshot	Statistical Analysis+ LSTM-RNN	Graph-level	Jedh et al. (2021)	2021
		Sketching	Statistical Analysis	Graph-level	Islam et al. (2022)	2022
			DGCNN	Graph-level	Zhang et al. (2023a)	2023

* The methods presented in King and Huang (2023); Messai and Seba (2023) offer a framework designed to accommodate the dynamic variations in data, allowing for a variety of encoder models to be employed, such as GCN, GAT and GraphSAGE.

tasks. In graph theory, a line graph or covering graph is a derivative graph constructed from an original graph wherein each vertex of the line graph corresponds to an edge of the original graph. Adjacency between two vertices in the line graph is established if and only if their corresponding edges in the original graph share a common vertex. Converting to a line graph facilitates mutual information flow between traffic from the same source node.

Chang and Branco (2021) constructed a bipartite graph with separate sets of source and target nodes and then transformed it into a corresponding line graph. This transformation addresses the issue of disjoint traffic from source nodes in the bipartite graph, which hinders direct information propagation. Differing from the model proposed by Chang and Branco (2021), GBA-IDS (Zhu and Lu, 2022) implemented a linear variant of GraphSAGE, called Line-GraphSAGE, to learn edge embeddings within their line graph.

However, the transformation into line graphs may incur significant computational costs and may not adapt well to all graph types (e.g., heterogeneous graphs), potentially generating redundant edges. Efforts have been made to improve and address this issue. Friji et al. (2023) proposed a novel graph construction method that models flows as nodes, linking flows with common IP origins or destinations. Deng et al. (2023) formulated three rules to construct the interval-constrained traffic graph (ICTG), considering three types of traffic transmission and time intervals to establish connections. The first rule pertains to traffic flows F_i and F_j sharing a common source IP; the second involves flows where the destination IP of F_i matches the source IP of F_j ; the third rule applies a temporal constraint, establishing links only between flows F_i and F_j with a time interval less than a predefined threshold T .

These graph construction methodologies offer the advantage of not relying directly on IP addresses or ports. Since attackers frequently utilize virtual private networks (VPNs) to obscure their true IP addresses, this strategy somewhat mitigates this issue. When the detection of attacks or anomalies is conceptualized as a node classification task, it essentially involves classifying all traffic emanating from a source IP. Consequently, once malicious traffic is identified, its source node is labeled as malicious, and all outgoing traffic from it is deemed to be of a malicious nature. Traffic forwarding systems, such as switches, would then discard it.

In terms of the second issue, one strategy is to enrich the original graph structure by adding edges and hyperedges, such as links between two-hop neighbors. For instance, Zheng and Li (2019) proposed a graph construction method (GCN-TC) different from the original one. To accommodate GCN models based on node classification, GCN-TC represented network flows as nodes in the graph and adds edges between flows with common sending or receiving nodes. Furthermore, they introduced a K-hop adjacency model, allowing the model to integrate information from neighbors beyond the one-hop neighbors.

Xiao et al. (2020) constructed first-order and second-order graphs to learn features from a global perspective.

The first-order graph considers communication between all hosts and ports as edges, learning latent features at the level of individual hosts. The second-order graph distinguishes between sender and receiver IPs and ports, considering their communication as edges. This approach learns two-hop data features, acquiring latent features from a global perspective.

In the TPE-NIDS framework (Zhang et al., 2022b), to surmount the constraints of random sampling, the degree-based Top-K Split-hop sampling algorithm was introduced. A one-dimensional ranking vector y is computed using a trainable mapping vector p and the node's feature matrix u . The top k values from the vector are selected as the sampling indices for the original node, as delineated by the following equation:

$$y_i = \frac{u_i p}{\|p\|}. \quad (2)$$

Beyond integrating information from multi-hop neighbors, another strategy is to extract additional information from the original features to serve as attributes for nodes and edges. Alert-GCN (Cheng et al., 2021) utilized one-hot encoding and similarity measures for graph construction, while NF-GNN (Busch et al., 2021) extracted directed node-based and edge-based representations from network flows. More specifically, NF-GNN extracted directed graph from a set of flows where nodes correspond to all endpoints involved in the flows. For all communicating pairs (source IP, target IP), a directed edge is added. The representation of the edge is obtained by aggregating all domain representation. Each feature has several attributes, including mean, standard deviation, skewness, and kurtosis, from which aggregated values are computed. The resulting graph captures the distribution of network traffic over a specific time frame, providing a richer representation of communication relationships compared to naive graph construction methods.

Rather than determining whether to add edges between nodes based on original data packets, Pujol-Perich et al. (2022) proposed using the degree of similarity as the criterion. In their model, each node represents a network flow, and each edge signifies the similarity between two flows, with cosine similarity employed as the edge weight. A threshold, determined through cross-validation, is applied to decide whether to establish an edge, thereby refining the graph structure. Similarly, Wang et al. (2023) took into consideration behavioral similarity to improve graph construction methods, proposing the BS-GAT model. In this model, data flows are viewed as nodes within the graph, with the behavioral rules of nodes serving as the edges in the graph. BS-GAT equated traffic behavior with human behavior observed in social networks, assigning different edge weights to different types of flows. Ultimately, this results in the construction of a graph where each node has a relatively uniform number of neighbors.

In addition to classifying information within data manually based on expert knowledge, an increasing number of studies are exploring the integration of machine learning and deep learning methods. These methods were used to extract

additional information that can be added to the attributes of nodes or edges. For instance, Zhang et al. (2023b) introduced a continuously iterative network reconstruction technique in their graph intrusion detection (GID) framework. They simultaneously considered the construction of the graph and the parameter learning of the GNN model as part of the model training. This approach not only facilitated the creation of a graph structure that closely aligns with the true data distribution (based on some prior information) but also refined and updated the graph structure through further learning from graph embeddings. Specifically, GID employs a multi-head weighted cosine similarity network learning method to obtain the network structure A of the input data X . The similarity is calculated as follows:

$$s_{ij}^k = \cos(b_k^l \odot v_i^l, b_k^l \odot v_j^l), \quad (3)$$

$$s_{ij} = \frac{1}{m} \sum_{k=1}^m s_{ij}^k, \quad (4)$$

where $\odot$ represents element-wise multiplication, and the average similarity s_{ij} is the mean of the similarities calculated by the m heads of weight vectors. In Eq.(4), v_i^l and v_j^l represent the vectors of nodes i and j at the l -th node embedding level, respectively. b_k represents the k -th weight vector (the k -th head) that will be updated during the training process. If $k = 1$, then v^l is the original features of the nodes.

To make the network structure closer to the real data distribution, the authors also used regularization techniques to design the network structure's loss function. This function aims to make different types of data sparsely connected or even not connected. The authors designed the network regularization structure loss as Eq.(5) by improving the l_1 -norm and the Dirichlet energy function.

$$\begin{aligned} L_N &= \alpha \Omega(A, X) + f(A) \\ &= \alpha \Omega(A, X) - \frac{\beta}{n} \mathbf{1}^T \log(A \mathbf{1}) + \frac{\gamma}{n^2} \|A\|_F^2, \end{aligned} \quad (5)$$

where α, β, γ are non-negative hyperparameters, and $\mathbf{1}$ represents an all-ones matrix. This equation represents the overall loss, which includes the sum of network smoothness, connectivity, and sparsity losses. The first term is the Dirichlet energy, which ensures the smoothness of the network. The second term applies the logarithmic barrier to vector $A \mathbf{1}$ to ensure the positivity of the degrees and the connectivity of the network. The last term calculates the Frobenius norm of the adjacency matrix in the network to control its sparsity.

However, systems based on homophily and structural equivalence (Bowman et al., 2020), which are constrained by homogeneous graph models, cannot fully exploit the network or host environment. In real-world scenarios, heterogeneous graphs, which encompass a variety of node and edge types, offer a more robust framework for intrusion detection, particularly when operating under real-time constraints with limited data.

Sun and Yang (2022) constructed a heterogeneous graph utilizing diverse types of log files, encapsulating a range

of network entities such as users, devices, and processes. They specify meta-paths as edges, to represent complex relationships among these entities. These meta-paths consist of four core internal network elements and six relationship types.

Existing methods generally deal with either traffic data or host data in isolation. NE-GConv (Altaf et al., 2023) integrated both host and flow features within IDS, specifying a graph structure capable of handling both node and edge features simultaneously. In this model, IoT sessions are represented as nodes using a combination of IP addresses and port numbers. The features of these nodes are host-specific attributes like payload fields. Inter-entity communications are represented as edges, with selected flow field attributes such as duration, packet count, and arrival time serving as edge features.

4.1.2. Dynamic Graph

Dynamic graphs, which evolve over time, are crucial for real-time intrusion detection. Static graphs are insufficient for streaming settings where data and host logs change dynamically. Most graph-based network traffic detection tools aim to analyze past traffic to understand previous attacks. Researchers are contemplating how to make graphs reflect real-time changes as the data evolves.

One strategy is to generate snapshots of the dynamic environment, that is constructing new graphs within a certain time frame, allowing the graphs to approximate and describe the full picture of the data. AddGraph (Zheng et al., 2019) constructed dynamic graphs that change over time. To address the issue of imbalanced anomaly data, AddGraph generated anomaly samples based on each edge, treating them as anomaly edges. CAN-IDS (Jedh et al., 2021) constructed dynamic message-sequence graphs, transforming message sequences within specified time intervals into directed graphs. In these message-sequence graphs, node labels represent CAN IDs, while edge labels represent the number of message transmissions between CAN IDs. Furthermore, these graphs change along with the timestamp t . Islam et al. (2022) also segmented CAN bus messages into multiple windows, constructing graphs for the messages within each window, and then attempted to discern the connections between each window.

Duan et al. (2023) captured the contextual evolution of communications between IP pairs through continuous snapshots, converting network traffic into a series of spatiotemporal graphs. King and Huang (2023) viewed interactions occurring within discrete units of time in the network as a series of snapshots, represented by graph G_t . In this representation, V is the set of entities in the network that have interactions, and $E_t = \{(u, v) \in V\}$ represents interactions occurring within time interval t .

While some works have considered real-time graph representations in networks, these are typically used for monitoring snapshots of the entire network, which can be resource-intensive. In the context of flow graphs, sketching

is another technique for dynamic graph representation, projecting high-dimensional data into low-dimensional feature vectors.

Messai and Seba (2023) introduced a graph-based framework that utilizes graphs to depict activity windows in network traffic, subsequently classifying the traffic as normal or anomalous. Similar to other approaches, this framework operates in two phases: initially generating graph structures based on all available attributes in traffic communication, followed by training GNNs to learn graph embeddings. Uniquely, each edge in the graph carries a timestamp, recording dynamic network communications. This allows for the dynamic addition and removal of vertices and edges as communications evolve over time.

Paudel and Eberle (2020) proposed an unsupervised, general-purpose graph sketching technique for intrusion detection within streaming setting environments. This technique conceptualizes network entities—such as DNS servers, web servers, workstations, and devices with internet connectivity—as nodes within a flow graph. The interactions between these entities are depicted as edges. In the context of dynamic graphs, traffic for each continuous fixed time period is represented as an individual graph.

The controller area network (CAN) serial communication protocol bus, integral to real-time applications, necessitates robust real-time intrusion detection systems. Existing real-time detection mechanisms are either limited to identifying a narrow spectrum of attack types or require an extensive volume of CAN messages during the detection process, thereby complicating the implementation of real-time functionalities. Zhang et al. (2023a) developed a CAN bus detection system capable of detecting all known attacks in as short as 3 milliseconds. This system involves constructing directed graphs of CAN messages over specified time intervals, with node features representing data content in CAN messages and edge features representing the frequency of typical CAN ID pairs.

Guo et al. (2022) argued that such models might overlook changes in data topology connections. To address this, they proposed GLD-Net, a model adept at learning both flow and topology features from time-series stream data. GLD-Net initially generates dynamic DDoS topology graphs from time series, a process divided into two phases: the initial phase converts traffic data into node or edge features, with traffic within certain time intervals forming distinct [IP-Feature] pairs stored in a feature table. The subsequent phase involves the construction of connection graphs, adding feature attributes to nodes or edges based on the feature table, and deciding whether to add these attributes based on whether two nodes have communication (flows) between them. This results in topological feature maps on m sub-time intervals ($G = G_1 \cup G_2 \cup \dots \cup G_m$). In GLD-Net, feature attributes include not only statistical data but also the final graph's connection status, packet tags, and centrality features.

Given resource constraints in streaming settings, there are insufficient resources for graph transformations, particularly when compared to systems based on static graphs. Dynamic graph constructions often incorporate additional information to enrich the graph, enabling models to learn data dynamics within the constraints of time and resources.

4.2. Graph-to-Feature in Network Intrusion Detection

Given a well-constructed graph, a challenge lies in defining the information propagation and fusion strategies within the GNNs (Xiao et al., 2021). The overall workflow for extracting node-based and edge-based representation from a graph is shown in Fig. 3.

With node content, edge attributes and graph structures serving as inputs for GNNs, their outputs can be tailored to accomplish various graph analysis tasks through one of the following three mechanisms:

- **Node Level:** Outputs at the node-level focus on the attributes or features of individual nodes, typically employed for tasks such as node classification, regression, or label prediction, where the output for each node is independent.
- **Edge Level:** Edge-level outputs concentrate on the attributes or relationships of edges within the graph, suitable for predicting or classifying connections between nodes, as in link prediction or edge classification tasks.
- **Graph Level:** Outputs at the graph-level are concerned with the attributes or features of the entire graph. This mechanism is utilized for classification or regression tasks related to the whole graph, such as graph classification, graph regression, or graph similarity assessment tasks.

4.2.1. Node-Level

GraphSAGE (Hamilton et al., 2017), as an early extension of GCN (Kipf and Welling, 2017), was the first method designed for information propagation within large graphs. This was a significant advancement for network intrusion detection systems (NIDS), especially considering that the graph structures derived from traffic data are typically extensive. However, the primary objective of NIDS is to identify anomalous network traffic, framing the issue predominantly as an edge classification problem, since that traffic information is integrated into the edge attributes of the graph. This makes it challenging for GCN and GraphSAGE, which are used for node and graph-level tasks, to function effectively.

As described in Section 4.1, many studies start with graph construction methods, transforming the problem into a node task while also utilizing edge information. GCN-TC (Zheng and Li, 2019) transformed the graph and subsequently used GCN for node classification.

Chang and Branco (2021) converted the original edge classification task into a node classification task by constructing a line graph. They introduced enhanced E-GraphSAGE and E-ResGAT algorithms. The introduced E-GraphSAGE learns extensive neighborhood information by sampling nodes within K hops. The E-ResGAT method integrates

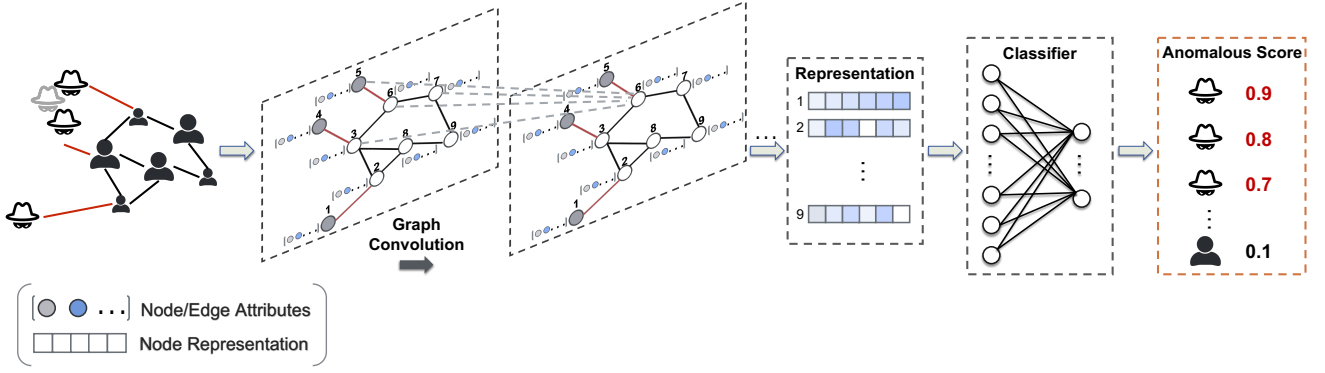


Figure 3: A Toy Example of Graph Convolution for Node or Edge-Level Intrusion Detection (Static Graph). The graph convolutional layer is employed to learn representations for nodes or edges. Classifiers such as fully connected networks (FCN) or multi-layer perceptrons (MLP) are trained to assign anomaly scores. The top-k data points are delineated as anomalies.

residual learning into the GNN, enabling the leveraging of multi-layer information within the graph. To consider the entire neighborhood collectively, the graph attention mechanism is also incorporated into the method.

Guo et al. (2022) employed graph attention networks (GAT) (Veličković et al., 2018) to learn fused features of the topology graph, which include both topological structural features and flow features. The output is a spatial sequence over the domain. Subsequent to the GAT, a long short-term memory (LSTM) network is utilized to mine relationships within the sequence. A fully connected layer and softmax function are then applied for classification. Wang et al. (2023) conducted GAT operations on graphs constructed based on behavioral similarity, assigning different weights to different edge behavior relationships.

Li et al. (2023) also made use of GAT and LSTM networks, proposing a multi-view contrastive learning (Multi-View CL) approach. Specifically, they devised a dual-view capture module to simultaneously learn graph-structured data and sequence data. Unlike GLD-Net, the Multi-View CL was conducted in parallel and obtains high-quality representations of multiple views through contrastive learning. The representation for each node after each GAT layer is denoted as:

$$h_v^{t+1} = \rho \left(\sum_{u \in \mathcal{N}_v} \alpha_{vu} h_u \right), \quad (6)$$

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{w}^T [h_v \parallel h_u]))}{\sum_{k \in \mathcal{N}_v} \exp(\text{LeakyReLU}(\mathbf{w}^T [h_v \parallel h_k]))}, \quad (7)$$

where $\mathbf{h}$ represents the hidden representation of a node. α is the attention score. $\mathbf{w} \in \mathbb{R}^{2D}$ is the weight matrix multiplied by the feature matrix, where D represents the embedding dimension of the node, and LeakyReLU is the nonlinear activation function.

Deng et al. (2023) introduced a network that concurrently learns both topology and statistical information. Building upon the foundation of the TAGCN network (Du et al., 2017), it incorporates a node-level spatial attention mechanism, aiming to extract features of key nodes.

4.2.2. Edge-Level

Addressing edge classification challenges necessitates not only improved graph construction methods but also enhancements to network models to fully exploit edge information. The objective is to ensure that the model's final output is edge embeddings, rather than node embeddings, to better capture the nuances of network interactions.

Lo et al. (2022) observed that the message-passing functions in traditional GraphSAGE algorithms rarely take into account features other than node features, which is insufficient for improving detection methods based on these models. In response, they introduced E-GraphSAGE for network intrusion detection in IoT networks. The distinction between E-GraphSAGE and the original GraphSAGE is that the former incorporates edge features into its inputs for edge feature fusion. E-GraphSAGE employs a specific node aggregation function and allows capturing both the edge features of graphs as well as the topological information.

GCN, with its simple, scalable architecture and computational efficiency, has become one of the most commonly used methods. AddGraph (Zheng et al., 2019) leverages both GCN and attention mechanisms to effectively learn information from dynamic graphs under a sequential model, as shown in Fig. 4.

Furthermore, E-minBatch (Lan et al., 2022) pre-samples nodes to obtain graph structures, allowing E-minBatch GraphSAGE to adapt to complex network structures. Compared to the original E-GraphSAGE model, E-minBatch can handle large-scale data inputs more effectively.

Caville et al. (2022) note that most GNN-based intrusion detection methods rely heavily on labeled traffic, which limits the scale and structure of input data. These methods also depend on the use of node features and neglect important edge information, leading to decreased detection accuracy. They address this by randomly shuffling the edges in real networks to create corrupted network graphs. This is done by shuffling the edge feature matrix in row X . Anomal-E then uses edge information and learns edge embeddings in a self-supervised manner, capturing key traffic features. The

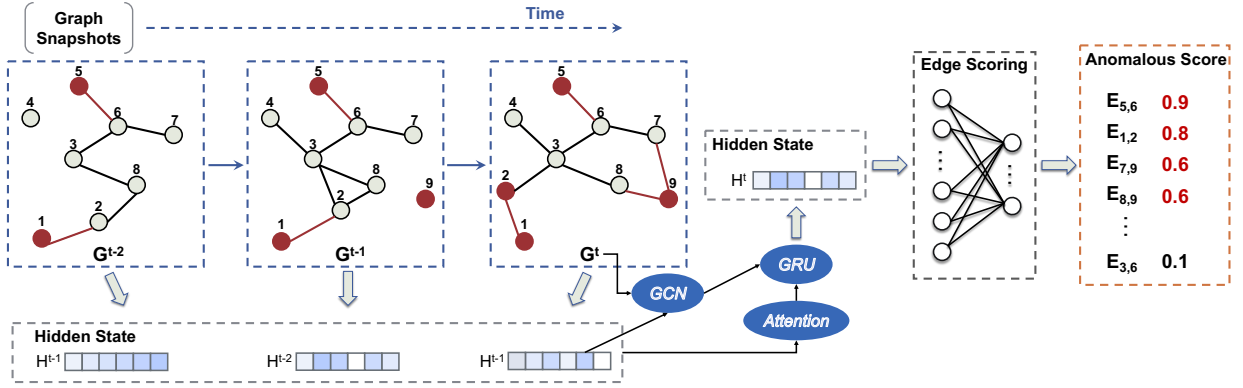


Figure 4: A Toy Example of Graph Convolution for Node or Edge-Level Intrusion Detection (Dynamic Graph). For each graph snapshot, the GCN is utilized to derive node embeddings from the temporal graph at discrete time intervals. Gated recurrent unit (GRU) synthesizes the current hidden state by integrating these node embeddings with preceding hidden states. The edge scoring function such as FCN is trained to allocate anomalous scores. The top-k data points are delineated as anomalies.

entire process uses E-GraphSAGE to extract both the edge embeddings and the corrupted edge embeddings.

HetGLM (Sun and Yang, 2022) provides an anomaly edge detection algorithm based on heterogeneous graphs. HetGLM fundamentally utilizes the CGNN (Zhang et al., 2022a) model framework, consisting of an encoder and a decoder, which together differentiate between anomalous and normal edges. Within the encoder, both fully connected network (FCN) and GNN encoder are employed to learn the relationships between nodes. The FCN is used to learn the attribute features of edges, while the GNN is utilized to generate embeddings for user and device nodes. Distinct from CGNN, HetGLM leverages the diversity of heterogeneous graphs by employing a meta-path-based random walk strategy for neighbor node sampling. This approach, coupled with the use of attention mechanisms during the aggregation of node attributes, aids in capturing the variations between different meta-path samplings. The aggregation operation is denoted as:

$$w^{p_j} = \frac{1}{|V_A|} \sum_{v \in V_A} \alpha_A^T \cdot \tanh(W_A \cdot h_v^{p_j} + b_A),$$

$$h_v = \sum_{p \in P_A} \frac{\exp(w^{p_j})}{\sum_{j \in |P_A|} \exp(w^{p_j})} \cdot h_v^{p_j}.$$
(8)

where V_A represents the set of nodes in type A. α_A , W_A , b_A are the weight matrices, $h_v^{p_j}$ represents the node embedding vector obtained through meta-path P_j , and P_A is the set of meta-paths.

NE-GConv (Altaf et al., 2023) is an enhancement of the GCN model. This model has two layers of GCN, used to learn host and network features in the graph structure separately. Ultimately, the edges are classified as either normal or malicious. DLGNN (Duan et al., 2023) employs dynamic GNNs to extract spatial information from each discrete snapshot, capturing the contextual evolution of communications between IP pairs through consecutive snapshots. Specifically, DLGNN integrates GCNII (Chen et al.,

2020) layers to extract spatial information within line graph structures, where GCNII, compared to the vanilla GCN, mitigates overfitting. Furthermore, it incorporates GCNII-GRU layers to capture the temporal information of discrete snapshots.

4.2.3. Graph-Level

In general, relatively scarce in the literature involves graph-level algorithms for network intrusion detection. This scarcity can be attributed to the nature of intrusion detection tasks, which often focus on identifying anomalous activities at the node or edge levels, such as unusual traffic patterns or suspicious connections between nodes. Graph-level analysis, on the other hand, would entail examining the entire network graph for signs of anomalous behavior, which is a more complex and computationally intensive task.

BotMark (Wang et al., 2020) takes into account both graph-based traffic behavior and statistically-based traffic behavior, as shown in Fig. 5. To identify Bot anomalies, they perform a hybrid analysis of these behavioral features, including anomaly analysis of the ego-net (a subgraph centered around a single node), stability analysis of the data flow, and similarity analysis. Rather than constructing a visual graph structure, they directly extract features based on the graph.

SNAPSKETCH (Paudel and Eberle, 2020) is an unsupervised graph representation technique designed for stream graphs, and the method process is shown in Fig. 6. This technique utilizes the node2vec (Grover and Leskovec, 2016) to generate snapshots of data streams at any given time t , embedding high-dimensional graph-structured data into low-dimensional vectors. It then employs Simplified Hashing for sampling to retain low-storage, low-dimensional sketch vectors. The main design principle of SNAPSKETCH is that the graph structure of anomalous graphs undergoes a sudden change rather than a slow evolution. The algorithm is adept at detecting such anomalies, proving helpful in identifying

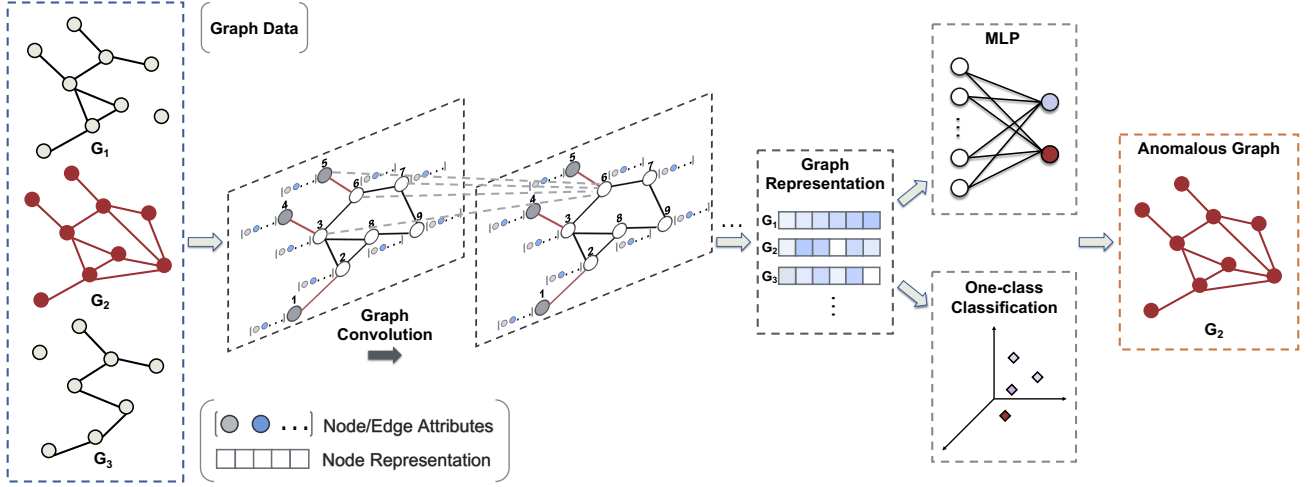


Figure 5: A Toy Example of Graph Convolution for Graph-Level Intrusion Detection (Static Graph). Node features aggregation in graph convolution layers. A global graph embedding is derived through pooling layer. By generating representation for each single graph, anomaly graphs can be detected by MLP or One-class classification.

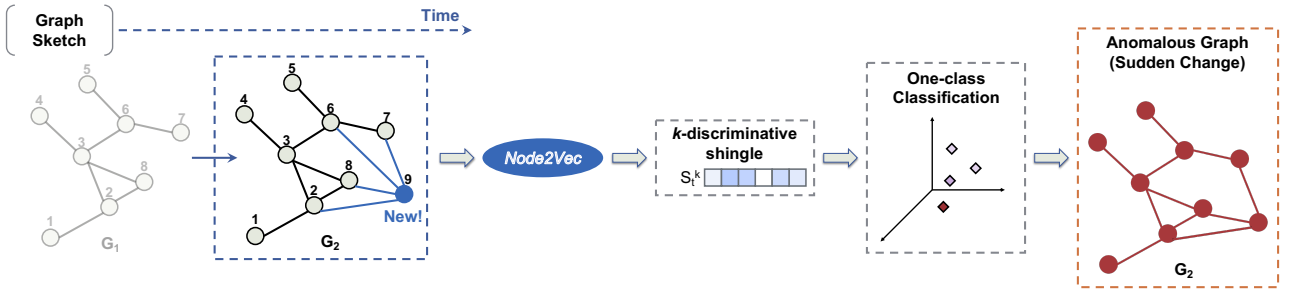


Figure 6: A Toy Example of Graph Convolution for Graph-Level Intrusion Detection (Dynamic Graph). For graph sketch, the Node2Vec computes its k -discriminative shingle S_i^k . By generating representation for a graph sketch, anomalous graphs can be detected using one-class classification; a graph is deemed anomalous if it exhibits a sudden change.

network attacks within computer networks, such as port scanning and denial-of-service attacks.

In the GLAD-PAW (Wan et al., 2021), it is recognized that the same communication event might occur at different positions within a session. However, after basic graph construction, this positional information is lost. Therefore, the model keeps track of the positions where each communication event occurs and incorporates position information layer (PAGAT) with learnable embeddings. In the model, the position-aware node embeddings are denoted as:

$$\mathbf{h}_{s,i} = \mathbf{h}_{v_{s,i}} + \mathbf{h}_{p_{s,i}}, \quad (9)$$

where $\mathbf{h}_{v_{s,i}}$, $\mathbf{h}_{p_{s,i}}$ are feature embeddings and position embeddings for i -th node in session s .

Additionally, the model incorporates a weighted graph attention layer (WGAT) to capture the entire neighborhood information of nodes. The model outputs a graph-level representation vector, denoted as:

$$\mathbf{h}_{G_s} = \tanh \left(\sum_{i=1}^{|V_s|} \text{softmax} \left(f \left(\mathbf{h}_{s,i}^L \right) \right) \odot \tanh \left(g \left(\mathbf{h}_{s,i}^{LL} \right) \right) \right). \quad (10)$$

The CAN bus intrusion detection method proposed by Zhang et al. (2023a) is based on graph-level algorithms. This method examines the CAN message graph within a certain time interval to determine whether the network environment has been attacked during this period. To address the issue of highly imbalanced training data, a two-tier classifier cascade was developed, consisting of a binary classifier and a multi-class classifier. The system first detects attacks from the constructed graph, and once an attack is detected in the first-tier classifier, the sample is sent to the second-tier classifier for attack type classification. The model employs DGCNN (Zhang et al., 2018) to learn the graph structure. The graph convolution process is as follows:

$$\mathbf{Z} = f \left(\tilde{\mathbf{D}}^{-1} \mathbf{A} \mathbf{X} \mathbf{W} \right). \quad (11)$$

To adapt it for CAN message graphs, the authors made modifications to the original network. The input feature matrix simultaneously includes both node and edge matrices, represented as $X_{n \times (n+n')}$. Since CAN message graphs allow self-loops, $\bar{A} = A + I$, where the addition of the identity matrix I supports self-loops.

4.3. Graph-to-Feature in Host Intrusion Detection

Compared to network intrusion detection, host intrusion detection system (HIDS) offers low-level and fine-grained information (including file systems, system calls, and network events) that serve as the foundation for detecting attacks that most network intrusion detection systems are unable to handle (Valeur et al., 2004). Effective graph representation is also crucial for building models capable of detecting complex attacks occurring at the host level. In this section, we will categorize our investigation into Provenance Graph and System Call Graph.

4.3.1. Provenance Graph

Provenance Graphs serve as a promising data source for host-based intrusion detection systems due to their ability to trace the origin and transformation of data (Hassan et al., 2020; Zipperle et al., 2022). These directed acyclic graphs (DAGs) visually or structurally represent relationships between data items, processes, and agents involved in data creation, modification, or usage. Understanding the history, lineage, and processing of data is vital for intrusion detection.

However, analyzing the provenance graph is challenging due to its large scale and the presence of redundant nodes. For instance, DeepHunter (Wei et al., 2021) extracts indicators of compromise (IoCs) from provenance graphs, incorporating them into node attributes to reduce redundancy. It utilizes GCN to generate node-level embeddings for the query graph and the provenance graphs, followed by employing the global context-aware attention strategy from SimGNN (Bai et al., 2019) to obtain graph-level embeddings for both graphs. Based on the embeddings h_{G_q} and h_{G_p} of the two graphs, DeepHunter utilizes a relational learning network (such as NTN (Socher et al., 2013)) to learn the relationship between the two graphs and outputs a graph matching score s . The model aims to minimize the difference between the actual matching score and the output matching score.

THREATTRACE (Wang et al., 2022) offers a multi-model framework for real-time host monitoring, utilizing a novel feature extraction and label assignment method for semi-supervised intrusion detection. The framework learns different features of benign data in a supervised mode, training only on subgraphs to ensure scalability. In a streaming setting, it maintains a dynamic queue Q of abnormal nodes, issuing alerts when the number of anomalies exceeds a predefined threshold $\hat{T}$, thereby providing a research point for reducing false positives in streaming settings. Accurately and timely detecting intrusions from massive multi-source logs presents a challenge for modern systems. Paradise (Wu

et al., 2023b) introduces a new extraction strategy, pruning and extracting process feature vectors from source dependencies at the system log level, and storing them in an efficient memory database. Paradise employs a multi-detector aggregation scheme, negotiating results from multiple detectors without additional communication overhead between detectors.

4.3.2. System Call Graph

System call graphs are integral for analyzing the behavior of applications, primarily focusing on the sequence and interaction of system calls during program execution. These graphs are essential for identifying abnormal or malicious activities within a system, providing insights into the runtime behavior of applications.

GRID (Hu et al., 2021) transforms system call sequences into a directed graph where each system call is represented as a node, and directed edges exist between adjacent system calls. Through a combination of random walks, word embedding techniques, and graph pooling, GRID generates more efficient sequence embedding representations. This approach facilitates a compact and expressive representation of system call sequences, capturing the inherent structure and patterns within the data, which is crucial for effective intrusion detection.

4.4. GNN Models Deployment

The diverse methods of data collection and storage make the design and deployment of IDS challenging. Most IDS placement strategies are suboptimal, resulting in their inefficiency (Bhale et al., 2023). Proposing reasonable solutions for optimal IDS placement is a consideration in many research works.

There are two main challenges: the integration level between models and the computational efficiency of the models. Many recent methods combine RNN and GNN to capture both topological and temporal features of data. However, these methods either depend on the output of RNNs during the GNN graph embedding phase or simply integrate GNN into the RNN architecture without accounting for their mutual interactions. This straightforward integration requires the models to be sequential, constraining scalability for extensive data. Another issue is that most of the memory consumption in existing architectures is spent on message passing within GNN, meaning subsequent simple models must wait for GNN computation to complete, leading to low efficiency.

GraphNET (Protogerou et al., 2021) presents a distributed deployment strategy for GNN models. This strategy involves deploying multi-agent systems across an entire internet of things (IoT) network, with each agent implementing a GNN model. To ensure connectivity throughout the IoT network, there is an exchange of information between each agent. GraphNET utilizes both edge multi-layer perceptron (MLP) and node MLP to extract data features.

Euler (King and Huang, 2023) permits several replicated GNNs to run independently on sequence encoders like RNNs, clearly enabling parallel execution of the GNN

embedding process. Through this approach, GNNs encode network topologies at discrete moments, while RNNs capture dynamically changing network connections.

4.5. Survey Objective

This section encapsulates the insights and findings derived from the systematic survey conducted on the application of graph neural networks in intrusion detection. Specifically, our goal is to answer the following research questions (RQs):

4.5.1. RQ1 (Sections 4.1): How are GNNs applied to intrusion detection?

Our survey indicates that GNNs are predominantly applied for supervised intrusion and anomaly detection within networks. Fewer works delve into exploring the application of GNNs for monitoring system, program, or user behavior within individual hosts.

4.5.2. RQ2 (Sections 4.2 and 4.3): What is the problem definition for intrusion detection based on GNNs?

The problem is typically defined as identifying abnormal traffic and potential security threats present within the network. This process is achieved by classifying unusual targets (such as nodes, edges, and graphs) through learning the inherent graph-structured features within network traffic. Alternatively, the emphasis might shift to detecting abnormal operations or accesses within a host. This is accomplished by training downstream classifiers on representations learned from provenance graphs and system call graphs of system entities. These classifiers are adept at distinguishing between normal and anomalous behavior, providing a robust mechanism for identifying and mitigating potential security risks.

4.5.3. RQ3: What considerations dominate when developing GNN-based intrusion detection?

Researchers focus on several key areas:

- **Graph Construction Methods:** Transforming implicit (inherently non-graph) graph structured data, such as network traffic and system calls, into a graph format suitable for GNN models is a primary concern. This involves determining the representation of nodes and edges, as well as extracting these representations from the raw data.

- **Design and Implementation of Learning Algorithms:** Developing new GNN learning algorithms to more accurately detect network intrusions is a focus area. These algorithms must be capable of handling large-scale graphs and effectively learning complex patterns within the graph structure.

- **Model Performance and Accuracy:** Improving model accuracy and reducing operational costs are crucial metrics for intrusion detection systems. Researchers are working to improve accuracy by enhancing the structure and learning algorithms of GNN models. Furthermore, considering that deployment environments often have limited resources,

there is a focus on making the models lightweight, which includes reducing the operational cost of running the models appropriately.

- **Practical Application and Deployment:** Optimizing the real-world deployment of trained GNN models for real-time intrusion detection. This involves computational efficiency and suitability of model for specific cyberspace environments.

4.5.4. RQ4: What are the prevailing trends in IDS based on GNNs?

- **Integration of Other Feature Learning Methods in GNNs:** Integrating other feature learning methods within GNNs. This integration aims to enhance the distinctiveness and robustness of the learned data representations.

- **Research on Dynamic Graph Algorithms in Stream Settings:** Exploring dynamic graph algorithms that operate effectively in stream settings to adapt to changes in real-time. With the advent of dynamic and ever-changing network environments, there is a need for models that can adapt and respond to these changes in real-time.

- **Development of Scalable and Efficient GNN Models:** Designing GNN models that are scalable and efficient. These models are designed to handle large-scale networks and high-dimensional data without compromising high detection accuracy or increasing false positive rates. The literature (Paudel and Eberle, 2020) provides a valuable starting point for researchers aiming to mitigate the challenge of false positives in streaming settings, and reducing the false-positive rate within a secure range would be a significant improvement.

5. Open Problems and Future Directions

5.1. User Information and Graph Data Privacy

While GNNs inherently process graph data, ensuring the privacy of user information within these structures remains a concern (Li et al., 2021a; Cai et al., 2021). To protect privacy, it is imperative to ensure that this data is anonymized or appropriately encrypted without compromising the graph's utility during analysis.

5.2. Model Interpretability

In large networks, especially when it comes to security decisions, the interpretability of models becomes crucial (Keshk et al., 2023; García-Sigüenza et al., 2023). Users and administrators need to understand why GNNs make specific predictions, which aids in building trust in the model and providing a logical background for decisions. Existing interpretability techniques might not be entirely suitable for GNN models, necessitating the development of new techniques or the adaptation of existing ones.

5.3. Model Robustness and Security

GNNs might be susceptible to adversarial attacks where attackers attempt to mislead the model by providing malicious inputs (Zhou et al., 2022). Therefore, there's a need

to develop robust models that can resist such adversarial attacks.

5.4. Efficient Integration and Analysis of Graph Data

Data from provenance graphs and system call graphs provide invaluable insights into host activities. However, the effective assimilation and analysis of this data present a series of challenges, from efficient graph representation to the reduction of redundancies. Future research in this area should aim to address these challenges, exploring innovative approaches and technologies to enhance the graph-to-feature process in host intrusion detection.

5.5. Heterogeneous Graph Neural Network

The challenge of achieving overall performance in real-world graph data with varying levels of homogeneity has been gaining more attention in the development of efficient architectures (Lee and Kim, 2022). Associating events from different sources (such as workstations, servers, NIDS, HID, firewalls) offers a more comprehensive perspective on network security. Heterogeneous GNNs provide an effective approach for analyzing these data concurrently.

5.6. Dynamic Graph Algorithms for Streaming Data

Flow graph analysis offers a novel approach to analyze and understand network traffic. This method can help in better understanding the data flow within networks, leading to more effective detection and prevention of potential security threats.

6. Conclusion

The integration of graph neural networks into intrusion detection systems has marked a significant stride in the realm of network security. Their inherent ability to process and analyze graph data offers a unique advantage, especially when dealing with complex network structures and patterns. However, as with any emerging technology, graph neural networks have sparked much research and also face challenges in this domain. We systematically reviewed the fundamental concepts of intrusion detection and graph neural networks, then delved deeply into various aspects of applying graph neural networks to intrusion detection, including graph construction, network design, and model deployment methods. The insights and findings presented in this review provide a foundational framework and exhibition of work for future research. We also discussed open issues and future research directions in this field, including ensuring user data privacy and enhancing model interpretability and robustness, hoping to provide valuable references for scholars and engineers. Focusing on these challenges will not only ensure the effectiveness of graph neural networks but also adapt to the ever-changing landscape of network threats and security needs. The future of intrusion detection, supported by the capabilities of graph neural networks, looks promising. Continued research and innovation in this field

will undoubtedly pave the way for a more secure and resilient network environment.

Acknowledgements

This research work was supported by National Natural Science Foundation of China under Grant No. 62272103, Natural Science Foundation of Fujian Province of China under Grant Nos. 2022J06020 and 2023J02014, Young Top Talent of Young Eagle Program of Fujian Province, China under Grant No. F21E0011202B01.

References

- Abdel Wahab, O., 2022. Intrusion detection in the iot under data and concept drifts: Online deep learning approach. *IEEE Internet of Things Journal* 9, 19706–19716. doi:10.1109/JIOT.2022.3167005.
- Agiollo, A., Bardhi, E., Conti, M., Lazeretti, R., Losiouk, E., Omicini, A., 2023. Gnn4ifa: Interest flooding attack detection with graph neural networks, in: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), pp. 615–630. doi:10.1109/EuroSP57164.2023.00043.
- Alshamrani, A., Myneni, S., Chowdhary, A., Huang, D., 2019. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials* 21, 1851–1877. doi:10.1109/COMST.2019.2891891.
- Altaf, T., Wang, X., Ni, W., Liu, R.P., Braun, R., 2023. Ne-gconv: A lightweight node edge graph convolutional network for intrusion detection. *Computers & Security* 130, 103285. doi:https://doi.org/10.1016/j.cose.2023.103285.
- Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W., 2019. Simgnn: A neural network approach to fast graph similarity computation, in: Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, pp. 384–392. doi:10.1145/3289600.3290967.
- Basati, A., Faghih, M.M., 2022. Pdae: Efficient network intrusion detection in iot using parallel deep auto-encoders. *Information Sciences* 598, 57–74. doi:https://doi.org/10.1016/j.ins.2022.03.065.
- Bhale, P., Chowdhury, D.R., Biswas, S., Nandi, S., 2023. Optimist: Lightweight and transparent ids with optimum placement strategy to mitigate mixed-rate ddos attacks in iot networks. *IEEE Internet of Things Journal* 10, 8357–8370. doi:10.1109/JIOT.2023.3234530.
- Bilot, T., Madhoun, N.E., Agha, K.A., Zouaoui, A., 2023. Graph neural networks for intrusion detection: A survey. *IEEE Access* 11, 49114–49139. doi:10.1109/ACCESS.2023.3275789.
- Bowman, B., Laprade, C., Ji, Y., Huang, H.H., 2020. Detecting lateral movement in enterprise computer networks with unsupervised graph AI, in: 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020), pp. 257–268. URL: <https://www.usenix.org/conference/raid2020/presentation/bowman>.
- Buczak, A.L., Guven, E., 2016. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials* 18, 1153–1176. doi:10.1109/COMST.2015.2494502.
- Busch, J., Kocheturov, A., Tresp, V., Seidl, T., 2021. Nf-gnn: Network flow graph neural networks for malware detection and classification, in: Proceedings of the 33rd International Conference on Scientific and Statistical Database Management, pp. 121–132. doi:10.1145/3468791.3468814.
- Cai, H., Zheng, V.W., Chang, K.C.C., 2018. A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering* 30, 1616–1637. doi:10.1109/TKDE.2018.2807452.
- Cai, Z., Xiong, Z., Xu, H., Wang, P., Li, W., Pan, Y., 2021. Generative adversarial networks: A survey toward private and secure applications. *ACM Computing Surveys* 54. doi:10.1145/3459992.
- Caville, E., Lo, W.W., Layeghy, S., Portmann, M., 2022. Anomal-e: A self-supervised network intrusion detection system based on graph neural

- networks. *Knowledge-Based Systems* 258, 110030. doi:<https://doi.org/10.1016/j.knosys.2022.110030>.
- Chang, L., Branco, P., 2021. Graph-based solutions with residuals for intrusion detection: the modified e-graphsage and e-resgat algorithms. *arXiv:2111.13597*.
- Chen, M., Wei, Z., Huang, Z., Ding, B., Li, Y., 2020. Simple and deep graph convolutional networks, in: *Proceedings of the 37th International Conference on Machine Learning*, pp. 1725–1735. URL: <https://proceedings.mlr.press/v119/chen20v.html>.
- Chen, Y., Wu, L., 2022. *Graph Neural Networks: Graph Structure Learning*. Springer. chapter 14. pp. 297–321. doi:[10.1007/978-981-16-6054-2_14](https://doi.org/10.1007/978-981-16-6054-2_14).
- Cheng, Q., Wu, C., Zhou, S., 2021. Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. *IEEE Communications Letters* 25, 1564–1567. doi:[10.1109/LCOMM.2020.3048995](https://doi.org/10.1109/LCOMM.2020.3048995).
- Corona, I., Giacinto, G., Roli, F., 2013. Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues. *Information Sciences* 239, 201–225. doi:<https://doi.org/10.1016/j.ins.2013.03.022>.
- Creech, G., Hu, J., 2014. A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns. *IEEE Transactions on Computers* 63, 807–819. doi:[10.1109/TC.2013.13](https://doi.org/10.1109/TC.2013.13).
- Cui, P., Wang, X., Pei, J., Zhu, W., 2019. A survey on network embedding. *IEEE Transactions on Knowledge and Data Engineering* 31, 833–852. doi:[10.1109/TKDE.2018.2849727](https://doi.org/10.1109/TKDE.2018.2849727).
- Debar, H., Dacier, M., Wespi, A., 1999. Towards a taxonomy of intrusion-detection systems. *Computer Networks* 31, 805–822. doi:[https://doi.org/10.1016/S1389-1286\(98\)00017-6](https://doi.org/10.1016/S1389-1286(98)00017-6).
- Deng, X., Zhu, J., Pei, X., Zhang, L., Ling, Z., Xue, K., 2023. Flow topology-based graph convolutional network for intrusion detection in label-limited iot networks. *IEEE Transactions on Network and Service Management* 20, 684–696. doi:[10.1109/TNSM.2022.3213807](https://doi.org/10.1109/TNSM.2022.3213807).
- Dong, G., Tang, M., Wang, Z., Gao, J., Guo, S., Cai, L., Gutierrez, R., Campbell, B., Barnes, L.E., Boukhechba, M., 2023. Graph neural networks in iot: A survey. *ACM Transaction Sensor Networks* 19. doi:[10.1145/3565973](https://doi.org/10.1145/3565973).
- Du, J., Zhang, S., Wu, G., Moura, J.M.F., Kar, S., 2017. Topology adaptive graph convolutional networks. *CoRR abs/1710.10370*. URL: <http://arxiv.org/abs/1710.10370>, *arXiv:1710.10370*.
- Duan, G., Lv, H., Wang, H., Feng, G., 2023. Application of a dynamic line graph neural network for intrusion detection with semisupervised learning. *IEEE Transactions on Information Forensics and Security* 18, 699–714. doi:[10.1109/TIFS.2022.3228493](https://doi.org/10.1109/TIFS.2022.3228493).
- Fraihat, S., Makhadmeh, S., Awad, M., Al-Betar, M.A., Al-Redhaei, A., 2023. Intrusion detection system for large-scale iot netflow networks using machine learning with modified arithmetic optimization algorithm. *Internet of Things* 22, 100819. doi:<https://doi.org/10.1016/j.iot.2023.100819>.
- Friji, H., Olivereau, A., Sarkiss, M., 2023. Efficient network representation for gnn-based intrusion detection, in: *Applied Cryptography and Network Security*, pp. 532–554.
- Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X., Li, Y., 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1. doi:[10.1145/3568022](https://doi.org/10.1145/3568022).
- García-Sigüenza, J., Llorens-Largo, F., Tortosa, L., Vicent, J.F., 2023. Explainability techniques applied to road traffic forecasting using graph neural network models. *Information Sciences* 645, 119320. doi:<https://doi.org/10.1016/j.ins.2023.119320>.
- Gavel, S., Raghuvanshi, A.S., Tiwari, S., 2022. Maximum correlation based mutual information scheme for intrusion detection in the data networks. *Expert Systems with Applications* 189, 116089. doi:<https://doi.org/10.1016/j.eswa.2021.116089>.
- Gibert, D., Mateu, C., Planes, J., 2020. The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. *Journal of Network and Computer Applications* 153, 102526. doi:<https://doi.org/10.1016/j.jnca.2019.102526>.
- Gong, L., Cheng, Q., 2019. Exploiting edge features for graph neural networks, in: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9203–9211. doi:[10.1109/CVPR.2019.00943](https://doi.org/10.1109/CVPR.2019.00943).
- Grover, A., Leskovec, J., 2016. Node2vec: Scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 855–864. doi:[10.1145/2939672.2939754](https://doi.org/10.1145/2939672.2939754).
- Guo, W., Qiu, H., Liu, Z., Zhu, J., Wang, Q., 2022. Gld-net: Deep learning to detect ddos attack via topological and traffic feature fusion. *Computational Intelligence and Neuroscience* 2022, 4611331. doi:[10.1155/2022/4611331](https://doi.org/10.1155/2022/4611331).
- Guo, Y., Peng, Y., Hao, R., Tang, X., 2023. Capturing spatial-temporal correlations with attention based graph convolutional network for network traffic prediction. *Journal of Network and Computer Applications* 220, 103746. doi:<https://doi.org/10.1016/j.jnca.2023.103746>.
- Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems*, pp. 1025–1035. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7e9ea9-Paper.pdf, doi:[10.5555/3294771.3294869](https://doi.org/10.5555/3294771.3294869).
- Han, M., Feng, S., Chen, C.L.P., Xu, M., Qiu, T., 2019. Structured manifold broad learning system: A manifold perspective for large-scale chaotic time series analysis and prediction. *IEEE Transactions on Knowledge and Data Engineering* 31, 1809–1821. doi:[10.1109/TKDE.2018.2866149](https://doi.org/10.1109/TKDE.2018.2866149).
- Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems, in: *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1172–1189. doi:[10.1109/SP40000.2020.00096](https://doi.org/10.1109/SP40000.2020.00096).
- Hu, X., Gao, W., Cheng, G., Li, R., Zhou, Y., Wu, H., 2023. Toward early and accurate network intrusion detection using graph embedding. *IEEE Transactions on Information Forensics and Security* 18, 5817–5831. doi:[10.1109/TIFS.2023.3318960](https://doi.org/10.1109/TIFS.2023.3318960).
- Hu, Z., Liu, L., Yu, H., Yu, X., 2021. Using graph representation in host-based intrusion detection. *Security and Communication Networks* 2021, 6291276. doi:[10.1155/2021/6291276](https://doi.org/10.1155/2021/6291276).
- Huang, H., Guo, S., Li, C., Sheng, J., Wang, L., Li, J., Liu, J., Zhong, S., 2021. Two-tier graph contextual embedding for cross-device user matching, in: *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, pp. 730–739. doi:[10.1145/3459637.3482308](https://doi.org/10.1145/3459637.3482308).
- Hwang, K., Cai, M., Chen, Y., Qin, M., 2007. Hybrid intrusion detection with weighted signature generation over anomalous internet episodes. *IEEE Transactions on Dependable and Secure Computing* 4, 41–55. doi:[10.1109/TDSC.2007.9](https://doi.org/10.1109/TDSC.2007.9).
- Islam, R., Refat, R.U.D., Yerram, S.M., Malik, H., 2022. Graph-based intrusion detection system for controller area networks. *IEEE Transactions on Intelligent Transportation Systems* 23, 1727–1736. doi:[10.1109/TITS.2020.3025685](https://doi.org/10.1109/TITS.2020.3025685).
- Jain, M., Kaur, G., Saxena, V., 2022. A k-means clustering and svm based hybrid concept drift detection technique for network anomaly detection. *Expert Systems with Applications* 193, 116510. doi:<https://doi.org/10.1016/j.eswa.2022.116510>.
- Jamalipour, A., Murali, S., 2022. A taxonomy of machine-learning-based intrusion detection systems for the internet of things: A survey. *IEEE Internet of Things Journal* 9, 9444–9466. doi:[10.1109/JIOT.2021.3126811](https://doi.org/10.1109/JIOT.2021.3126811).
- Javed, Y., Khayat, M.A., Elghariani, A.A., Ghaffoor, A., 2023. Prism: A hierarchical intrusion detection architecture for large-scale cyber networks. *IEEE Transactions on Dependable and Secure Computing* early access, 1–17. doi:[10.1109/TDSC.2023.3240315](https://doi.org/10.1109/TDSC.2023.3240315).
- Jedh, M., Ben Othmane, L., Ahmed, N., Bhargava, B., 2021. Detection of message injection attacks onto the can bus using similarities of successive messages-sequence graphs. *IEEE Transactions on Information Forensics and Security* 16, 4133–4146. doi:[10.1109/TIFS.2021.3098162](https://doi.org/10.1109/TIFS.2021.3098162).
- Kaur, R., Gabrijelčič, D., Klobučar, T., 2023. Artificial intelligence for cybersecurity: Literature review and future research directions. *Information Fusion* 97, 101804. doi:<https://doi.org/10.1016/j.inffus.2023.101804>.

- Keshk, M., Koroniotis, N., Pham, N., Moustafa, N., Turnbull, B., Zomaya, A.Y., 2023. An explainable deep learning-enabled intrusion detection framework in iot networks. *Information Sciences* 639, 119000. doi:<https://doi.org/10.1016/j.ins.2023.119000>.
- King, I.J., Huang, H.H., 2023. Euler: Detecting network lateral movement via scalable temporal link prediction. *ACM Transactions on Privacy and Security* 26. doi:10.1145/3588771.
- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks, in: *International Conference on Learning Representations*, pp. 1–14. URL: <https://openreview.net/forum?id=SJU4ayYgl>.
- Ko, H., Praca, I., Choi, S.G., 2023. Anomaly detection analysis based on correlation of features in graph neural network. *Multimedia Tools and Applications* early access. doi:10.1007/s11042-023-15635-z.
- Lan, J., Lu, J.Z., Wan, G.G., Wang, Y.Y., Huang, C.Y., Zhang, S.B., Huang, Y.Y., Ma, J.N., Damaševičius, R., 2022. E-minbatch graphsage: An industrial internet attack detection model. *Security and Communication Networks* 2022, 5363764. doi:10.1155/2022/5363764.
- Landauer, M., Skopik, F., Frank, M., Hotwagner, W., Wurzenberger, M., Rauber, A., 2023. Maintainable log datasets for evaluation of intrusion detection systems. *IEEE Transactions on Dependable and Secure Computing* 20, 3466–3482. doi:10.1109/TDSC.2022.3201582.
- Lazzarini, R., Tianfield, H., Charissis, V., 2023. A stacking ensemble of deep learning models for iot intrusion detection. *Knowledge-Based Systems* 279, 110941. doi:<https://doi.org/10.1016/j.knsys.2023.110941>.
- Lee, M., Kim, S.B., 2022. Hapgnn: Hop-wise attentive pagerank-based graph neural network. *Information Sciences* 613, 435–452. doi:<https://doi.org/10.1016/j.ins.2022.09.041>.
- Li, K., Luo, G., Ye, Y., Li, W., Ji, S., Cai, Z., 2021a. Adversarial privacy-preserving graph embedding against inference attack. *IEEE Internet of Things Journal* 8, 6904–6915. doi:10.1109/JIOT.2020.3036583.
- Li, M., Huang, H., Zhao, L., Wang, Y., Sheng, H., Zhang, J., 2023. A sequence and graph contrastive learning based model for detecting cyber attacks behavior, in: *Proceedings of the 2022 5th International Conference on Machine Learning and Machine Intelligence*, pp. 160–165. doi:10.1145/3568199.3568225.
- Li, X., Zhu, M., Yang, L.T., Xu, M., Ma, Z., Zhong, C., Li, H., Xiang, Y., 2021b. Sustainable ensemble learning driving intrusion detection model. *IEEE Transactions on Dependable and Secure Computing* 18, 1591–1604. doi:10.1109/TDSC.2021.3066202.
- Liu, M., Xue, Z., Xu, X., Zhong, C., Chen, J., 2018. Host-based intrusion detection system with system calls: Review and future trends. *ACM Computing Surveys* 51. doi:10.1145/3214304.
- Liu, Y., Ding, K., Wang, J., Lee, V., Liu, H., Pan, S., 2023. Learning strong graph neural networks with weak information, in: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1559–1571. doi:10.1145/3580305.3599410.
- Lo, W.W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M., 2022. E-graphsage: A graph neural network based intrusion detection system for iot, in: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–9. doi:10.1109/NOMS54207.2022.9789878.
- Louk, M.H.L., Tama, B.A., 2023. Dual-ids: A bagging-based gradient boosting decision tree model for network anomaly intrusion detection system. *Expert Systems with Applications* 213, 119030. doi:<https://doi.org/10.1016/j.eswa.2022.119030>.
- Ma, X., Wu, J., Xue, S., Yang, J., Zhou, C., Sheng, Q.Z., Xiong, H., Akoglu, L., 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* early access, 1–1. doi:10.1109/TKDE.2021.3118815.
- Messai, M.L., Seba, H., 2023. Iot network attack detection: Leveraging graph learning for enhanced security, in: *Proceedings of the 18th International Conference on Availability, Reliability and Security*, pp. 1–7. doi:10.1145/3600160.3600503.
- Nguyen, T.T., Reddi, V.J., 2023. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems* 34, 3779–3795. doi:10.1109/TNNLS.2021.3121870.
- Paudel, R., Eberle, W., 2020. Snapsketch: Graph representation approach for intrusion detection in a streaming graph, in: *Proceedings of the 16th International Workshop on Mining and Learning with Graphs (MLG)*, p. null. doi:<https://doi.org/10.1145/1122445.1122456>.
- Pontarelli, S., Bianchi, G., Teofili, S., 2013. Traffic-aware design of a high-speed fpga network intrusion detection system. *IEEE Transactions on Computers* 62, 2322–2334. doi:10.1109/TC.2012.105.
- Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., Shyu, M.L., Chen, S.C., Iyengar, S.S., 2018. A survey on deep learning: Algorithms, techniques, and applications. *ACM Computing Surveys* 51. doi:10.1145/3234150.
- Protogerou, A., Papadopoulos, S., Drosou, A., Tzovaras, D., Refanidis, I., 2021. A graph neural network method for distributed anomaly detection in iot. *Evolving Systems* 12, 19–36. doi:10.1007/s12530-020-09347-0.
- Pujol-Perich, D., Suarez-Varela, J., Cabellos-Aparicio, A., Barlet-Ros, P., 2022. Unveiling the potential of graph neural networks for robust intrusion detection. *ACM SIGMETRICS Performance Evaluation Review* 49, 111–117. doi:10.1145/3543146.3543171.
- Qi, G.J., Luo, J., 2022. Small data challenges in big data era: A survey of recent progress on unsupervised and semi-supervised methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 2168–2187. doi:10.1109/TPAMI.2020.3031898.
- Qian, C., Yu, W., Lu, C., Griffith, D., Gollme, N., 2022. Toward generative adversarial networks for the industrial internet of things. *IEEE Internet of Things Journal* 9, 19147–19159. doi:10.1109/JIOT.2022.3163894.
- Ren, H., Tang, Y., Dong, W., Ren, S., Jiang, L., 2023. Duen: Dynamic ensemble handling class imbalance in network intrusion detection. *Expert Systems with Applications* 229, 120420. doi:<https://doi.org/10.1016/j.eswa.2023.120420>.
- Singh, A., Amutha, J., Nagar, J., Sharma, S., 2023. A deep learning approach to predict the number of k-barriers for intrusion detection over a circular region using wireless sensor networks. *Expert Systems with Applications* 211, 118588. doi:<https://doi.org/10.1016/j.eswa.2022.118588>.
- Socher, R., Chen, D., Manning, C.D., Ng, A.Y., 2013. Reasoning with neural tensor networks for knowledge base completion, in: *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1*, pp. 926–934. URL: https://proceedings.neurips.cc/paper_files/paper/2013/file/b337e84de8752b27eda3a12363109e80-Paper.pdf.
- Sun, J., Cheng, Z., Zuberi, S., Perez, F., Volkovs, M., 2021. Hgcf: Hyperbolic graph convolution networks for collaborative filtering, in: *Proceedings of the Web Conference 2021*, pp. 593–601. doi:10.1145/3442381.3450101.
- Sun, X., Yang, J., 2022. Hetglm: Lateral movement detection by discovering anomalous links with heterogeneous graph neural network, in: *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pp. 404–411. doi:10.1109/IPCCC55026.2022.9894347.
- Sun, Y., Zhu, D., Du, H., Tian, Z., 2023. Mhnf: Multi-hop heterogeneous neighborhood information fusion graph representation learning. *IEEE Transactions on Knowledge and Data Engineering* 35, 7192–7205. doi:10.1109/TKDE.2022.3186158.
- Umer, M.F., Sher, M., Bi, Y., 2017. Flow-based intrusion detection: Techniques and challenges. *Computers & Security* 70, 238–254. doi:<https://doi.org/10.1016/j.cose.2017.05.009>.
- Valeur, F., Vigna, G., Kruegel, C., Kemmerer, R., 2004. Comprehensive approach to intrusion detection alert correlation. *IEEE Transactions on Dependable and Secure Computing* 1, 146–169. doi:10.1109/TDSC.2004.21.
- Velicković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: *International Conference on Learning Representations*, pp. 1–12. URL: <https://openreview.net/forum?id=rJXmpikCZ>.
- Wan, Y., Liu, Y., Wang, D., Wen, Y., 2021. Glad-paw: Graph-based log anomaly detection by position aware weighted graph attention network, in: *Advances in Knowledge Discovery and Data Mining*, pp. 66–77. doi:https://doi.org/10.1007/978-3-030-75762-5_6.
- Wang, S., Wang, Z., Zhou, T., Sun, H., Yin, X., Han, D., Zhang, H., Shi, X., Yang, J., 2022. Threatrace: Detecting and tracing host-based threats

- in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security* 17, 3972–3987. doi:10.1109/TIFS.2022.3208815.
- Wang, W., Shang, Y., He, Y., Li, Y., Liu, J., 2020. Botmark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors. *Information Sciences* 511, 284–296. doi:https://doi.org/10.1016/j.ins.2019.09.024.
- Wang, Y., Han, Z., Li, J., He, X., 2023. Bs-gat: Behavior similarity based graph attention network for network intrusion detection. doi:https://doi.org/10.48550/arXiv.2304.07226, arXiv:2304.07226.
- Wei, R., Cai, L., Zhao, L., Yu, A., Meng, D., 2021. Deephunter: A graph neural network based approach for robust cyber threat hunting, in: *Security and Privacy in Communication Networks*, pp. 3–24. doi:10.1007/978-3-030-90019-9_1.
- Weller-Fahy, D.J., Borghetti, B.J., Sodemann, A.A., 2015. A survey of distance and similarity measures used within network intrusion anomaly detection. *IEEE Communications Surveys & Tutorials* 17, 70–91. doi:10.1109/COMST.2014.2336610.
- Wu, Y., Nie, L., Wang, S., Ning, Z., Li, S., 2023a. Intelligent intrusion detection for internet of things security: A deep convolutional generative adversarial network-enabled approach. *IEEE Internet of Things Journal* 10, 3094–3106. doi:10.1109/JIOT.2021.3112159.
- Wu, Y., Xie, Y., Liao, X., Zhou, P., Feng, D., Wu, L., Li, X., Wildani, A., Long, D., 2023b. Paradise: Real-time, generalized, and distributed provenance-based intrusion detection. *IEEE Transactions on Dependable and Secure Computing* 20, 1624–1640. doi:10.1109/TDSC.2022.3160879.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S., 2021. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 4–24. doi:10.1109/TNNLS.2020.2978386.
- Xiao, Q., Liu, J., Wang, Q., Jiang, Z., Wang, X., Yao, Y., 2020. Towards network anomaly detection using graph embedding, in: *Computational Science – ICCS 2020*, pp. 156–169. doi:https://doi.org/10.1007/978-3-030-50423-6_12.
- Xiao, T., Chen, Z., Wang, D., Wang, S., 2021. Learning how to propagate messages in graph neural networks, in: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 1894–1903. doi:10.1145/3447548.3467451.
- Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2019. How powerful are graph neural networks?, in: *International Conference on Learning Representations*, pp. 1–17. URL: https://openreview.net/forum?id=ryGs6iA5Km.
- Xu, P., Lu, G., Li, Y., Xu, C., 2023. Ee-gcn: A graph convolutional network based intrusion detection method for iiot, in: *2023 5th International Conference on Natural Language Processing (ICNLP)*, pp. 338–344. doi:10.1109/ICNLP58431.2023.00068.
- Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., Lin, X., 2023. Automatic evasion of machine learning-based network intrusion detection systems. *IEEE Transactions on Dependable and Secure Computing* early access, 1–16. doi:10.1109/TDSC.2023.3247585.
- Ye, N., Emran, S., Chen, Q., Vilbert, S., 2002. Multivariate statistical analysis of audit trails for host-based intrusion detection. *IEEE Transactions on Computers* 51, 810–820. doi:10.1109/TC.2002.1017701.
- Yu, L., Sun, L., Du, B., Liu, C., Lv, W., Xiong, H., 2023. Heterogeneous graph representation learning with relation awareness. *IEEE Transactions on Knowledge and Data Engineering* 35, 5935–5947. doi:10.1109/TKDE.2022.3160208.
- Yuan, H., Yu, H., Gui, S., Ji, S., 2023. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 5782–5799. doi:10.1109/TPAMI.2022.3204236.
- Zhang, G., Li, Z., Huang, J., Wu, J., Zhou, C., Yang, J., Gao, J., 2022a. Efraudcom: An e-commerce fraud detection system via competitive graph neural networks. *ACM Transactions on Information Systems* 40. doi:10.1145/3474379.
- Zhang, H., Zeng, K., Lin, S., 2023a. Federated graph neural network for fast anomaly detection in controller area networks. *IEEE Transactions on Information Forensics and Security* 18, 1566–1579. doi:10.1109/TIFS.2023.3240291.
- Zhang, M., Cui, Z., Neumann, M., Chen, Y., 2018. An end-to-end deep learning architecture for graph classification. *Proceedings of the AAAI Conference on Artificial Intelligence* 32. doi:10.1609/aaai.v32i1.11782.
- Zhang, Y., Yang, C., Huang, K., Li, Y., 2023b. Intrusion detection of industrial internet-of-things based on reconstructed graph neural networks. *IEEE Transactions on Network Science and Engineering* 10, 2894–2905. doi:10.1109/TNSE.2022.3184975.
- Zhang, Y., Zhang, Y., Wu, Y., Li, C., 2022b. Tpe-nids: uses graph neural networks to detect malicious traffic, in: *2022 4th International Conference on Frontiers Technology of Information and Computer (ICFTIC)*, pp. 949–958. doi:10.1109/ICFTIC57696.2022.10075261.
- Zhang, Z., Cui, P., Zhu, W., 2022c. Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* 34, 249–270. doi:10.1109/TKDE.2020.2981333.
- Zheng, J., Li, D., 2019. Gcn-tc: Combining trace graph with statistical features for network traffic classification, in: *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, pp. 1–6. doi:10.1109/ICC.2019.8761115.
- Zheng, L., Li, Z., Li, J., Li, Z., Gao, J., 2019. Addgraph: Anomaly detection in dynamic graph using attention-based temporal gcn, in: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*, pp. 4419–4425. doi:10.24963/ijcai.2019/614.
- Zhong, M., Lin, M., He, Z., 2023. Dynamic multi-scale topological representation for enhancing network intrusion detection. *Computers & Security* 135, 103516. doi:https://doi.org/10.1016/j.cose.2023.103516.
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M., 2020a. Graph neural networks: A review of methods and applications. *AI Open* 1, 57–81. doi:https://doi.org/10.1016/j.aiopen.2021.01.001.
- Zhou, J., Xu, Z., Rush, A.M., Yu, M., 2020b. Automating botnet detection with graph neural networks. *CoRR abs/2003.06344*. URL: https://arxiv.org/abs/2003.06344, arXiv:2003.06344.
- Zhou, X., Liang, W., Li, W., Yan, K., Shimizu, S., Wang, K.I.K., 2022. Hierarchical adversarial attacks against graph-neural-network-based iot network intrusion detection system. *IEEE Internet of Things Journal* 9, 9310–9319. doi:10.1109/JIOT.2021.3130434.
- Zhu, H., Lu, J., 2022. Graph-based intrusion detection system using general behavior learning, in: *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*, pp. 2621–2626. doi:10.1109/GLOBECOM48099.2022.10001299.
- Zipperle, M., Gottwalt, F., Chang, E., Dillon, T., 2022. Provenance-based intrusion detection systems: A survey. *ACM Computing Surveys* 55. doi:10.1145/3539605.